CPPDescent: A C++ library for the creation of K-NN graphs from multi-dimensional datasets

Software Development for Computing Systems, Winter 2023-2024

Department of Informatics and Telecommunications, University of Athens

Konstantinos Chousos
sdi2000215@di.uoa.gr

Anastasios-Phaedon Seitanidis
sdi2000179@di.uoa.gr

January 20, 2024

Abstract

This library is a C++ implementation of the "NN-Descent" algorithm by Dong, Moses, and Li [4], with a few improvements and optimisations (e.g. usage of random projection trees [3]). This implementation serves as an entry for the ACM SIGMOD 2023 competition. It was developed as part of the *Software Development for Computing Systems* course of the Department of Informatics and Telecommunications, taught in the winter of 2023 [9]. The source code is available at `https://github.com/kchousos/CPPDescent` [2].

1 Introduction

As is tradition for this course, our project for the semester was the challenge of last spring's ACM SIGMOD Student Research Competition. The theme of the competition was the efficient and fast creation of a *K Nearest Neighbor* graph, where its vertices belong to a dataset of 100-dimensions vectors/datapoints of floats.

A brute force approach to this problem has a complexity of $O(n^2)$, since each datapoint needs to check all others so that it can *weed out* the K points nearest to it. An answer to this problem was given in 2011 by Dong, Moses, and Li [4]. in their paper titled "Efficient K-Nearest Neighbor Graph Construction for Generic Similarity Measures" [4], where they present the *NN-Descent* algorithm. This algorithm is based at the very simple idea that *"a neighbor of a neighbor is also likely to be a neighbor"*. According to them, this algorithm is shown to have a complexity of $O(n^{1.14})$. This is a major step-up from the quadratic time, especially when it comes to big dataset.

Our library does not only implement the NN-Descent algorithm. It also implements other optimizations that e.g. make the starting graph more like the one we want to end up with, lowering this way the number of iterations the algorithm must do to reach a satisfying result. These optimizations are discussed in sec. 2.

1.1 Parameters

The full algorithm can be tweaked by a number of parameters. The ones that have to do with the NN-Descent algorithm are: K for the number of nearest neighbors to search for, δ for the threshold to the early termination [4, sec. 2.6] and ρ for the sampling rate [4, sec. 2.5].

Apart from those, there are two other parameters that have to do with the creation of the starting graph when random projection trees are used (sec. 2.2): D for the upper bound to the number of vertices each leaf must have and T for the number of random projection trees to compute.

1.2 Project layout

The library itself is stored in the `src/` directory. It is compiled to a `.a` file, so that it can be linked with any executable.

The main app that results to the main executable is in the `app/` directory. This file contains the only `main()` on the project, and can be thought as the API between the user and the library itself.

The directory `datasets/` contains a lot of the datasets of the SIGMOD competition in binary format. The subdirectory `computed/` contains some of the K-NN graphs that were computed using the brute force algorithm for comparison purposes. They are also in binary format. For which dataset and for which K they correspond to can be deduced from the respective filename.

The `docs/` subdirectory contains the `html/` and `latex/` subdirectories that they themselves contain files respective to each format. It is documentation of the code, automatically generated by code comments using Doxygen. The `lcov/` subdirectory contains files that are generated by LCOV and are used to monitor the code coverage provided by the tests (see below)[1],

The `extern/` directory contains the source code for the Google Test suite. It is included as a git submodule.

The `include/` directory contains header files for the library and the home-made data structures it uses.

The `test/` directory contains unit tests for the library.

The `helper.sh` file is a bash script that is used to automate some repetitive tasks, such as execution of all the tests, or documentation generation.

The `experiments.sh` is a script that executes the main program in quiet mode, and outputs a `.csv` file that contains the results of different executions for different parameters (see sec. 3).

1.3 Data structures

1.3.1 Defunct Structures

1.3.1.1 ADTMap using HashTable In the first and second submission we implemented the ADTGraph using an adjacency ADTMap, a map where we stored a pair of vertices (edge) as a key that was related with its weight. For the implementation of this ADTMap we used a hash table. In the final submission, we removed the map data type as we find a simpler and more efficient way to implement the graph data type.

1.3.1.2 ADTList using LinkedList In previous verisons we also used ADTList, implemented using Linked List to get access to some of the data stored in the mentioned graph. This data type is not used anymore as well, as we replaced it by the more efficient ADTPriorityQueue in some cases and ADTVector in other where having random access to the elements could be helpful.

1.3.2 Used Structures

1.3.2.1 ADTGraph In the final version the ADTGraph is implemented using a logic similar to the adjacency matrix. More specifically, in the graph is stored an ADTVector that contains its vertices each one of which are represented by a class called GraphVertex. In this class, we keep a PriorityQueue which contains the vertex's direct neighbors and one that contains its reverse neighbors.

1.3.2.2 ADTPriorityQueue using Heap As we mentioned earlier, we use a PriorityQueue which is implemented by a heap data structure. In this data type we have added some functionality by implementing a `remove` function that removes from the queue the node that contains a value equivalent to a given one. The search in this case has time complexity $O(n)$, worst-case.

1.4 Library interface

`Vector* readBinData(const char* fp, int dimensions)`: reads data from a binary file and stores it in a vector.

`void writeBinGraph(const char* fp, Graph* graph, int K)`: writes an already computed graph in a binary file.

`Graph* readBinGraph(const char* fp, int dimensions)`: reads and returns a graph from a binary file.

`float recall(Graph* bfGraph, Graph* nnGraph, int N, int K)`: returns the recall of the graph computed by NN-Descent, compared to the brute force graph.

`float euclideanDistance(Pointer a, Pointer b)`: metric function that computes the euclidean distance between 2 points.

`float manhattanDistance(Pointer a, Pointer b)`: metric function that computes the manhattan distance between 2 points.

`Graph* KNNBruteForceGraph(Vector* data, int K, CompareFunc compare)`: function that computes the K-NN graph using brute force.

`Graph* NNDescent_KNNGraph(Vector* data,int K,int D,int Trees,float delta,float rho,DistanceFunc distance)`: function that computes the K-NN graph for the given dataset using the NN-Descent algorithm.

`PQueue* NNDescent_Query(Graph* graph,int K,CompareFunc compare,Vector* query)`: function that computes the K Nearest Neighbors of the query point in the graph.

`void RPTree(Graph* graph, Vector* vec, int K, int D, int dimensions)`: creates a graph using random projection trees.

2 Optimizations

Since this project was developed in three stages, each stage called for new optimizations upon the code of the

[1] At the moment, the code coverage of the `cppdescent` library itself is pretty low, since it went very recently under a large refactoring and overhaul.

previous one. The second assignment/submission was focused on the optimizations proposed by Dong, Moses, and Li [4]. Namely *local join*, *incremental search*, *sampling* and *early termination*. Each of those is presented extensively on the NN-Descent paper, so there is no need to do the same here.

On the other hand, the third assignment called for some more interesting optimizations: By utilizing linear algebra, it is possible to simplify the computation of a distance between two vectors by a lot. Another optimization is a different approach on the creation of the starting graph, from which the NN-Descent algorithm starts running iteratively. Lastly, a very simple and obvious optimization is making use of the powerful computing systems of today and introducing parallelization/multi-threading to the algorithm.

2.1 Precomputed vector norms

Finding the distance between two points is a big chunk of this algorithm's computing time. Especially when we are talking about the *euclidean* distance for 100-dimensional points (eq. 1).

$$d_{\mathbf{x},\mathbf{y}} = \sqrt{(x_1 - y_1)^2 + \ldots + (x_N - y_N)^2} \tag{1}$$

The first thing we observe is that for the needs of the algorithm, the square root is unnecessary. The resulting expression can be then rewritten as vector operations, like so:

$$ds_{\mathbf{x},\mathbf{y}} = \mathbf{x}^2 + \mathbf{y}^2 - 2\mathbf{x}\mathbf{y},$$

where $\mathbf{x}^2$ and $\mathbf{y}^2$ are the square euclidean norms of the respective vectors. This gives us the option to compute these values only once for each vector and save them, saving precious computing time.

For the computation of the euclidean norms, the dot product of $\mathbf{x}\mathbf{y}$, but also for the representation of the vectors themselves the GNU Scientific Library [7] is used.

2.2 Random Projection Trees

Before the algorithm can begin improving a given graph, first a graph must be given. The default strategy is to draw K random edges for each vertex in the newly read graph, thus getting a totally random head-start.

A more sophisticated technique is the usage of random projection trees [3] for the initialization of said graph. The idea is simple: For the given dataset, we take a line (actually hyperplane for dimensions > 2) and split it in two halves. Then, we do the same thing recursively until we have at most D datapoints on each split — these final areas are the tree's *leaves*. The given dataset will then be splitted something like the one shown in fig. 1.

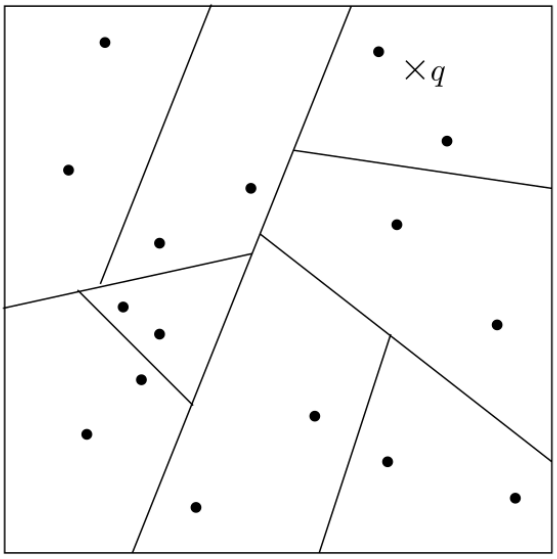

Figure 1: A 2-dimensional dataset partitioned by a random projection tree [3].

The only problem that this method brings is that there is a danger of the graph being non-connected. For this, we can do two things: Set D strictly lower than K and then add random neighbors until their number reaches K. Secondly, we can create more than one RPT. In this library, specifically in the `RPTree()` function, the RPTs are created recursively and concurrently, with each half of each split being "taken" by a new thread. This is done using the OpenMP Library [11], which is also used more generally in the program, as part of the next optimization: parallelization.

2.3 Parallelization

Portions of the algorithm are executed in parallel, since they access and modify different things. For this purpose, the OpenMP Library [11] is used, since it provides an easy and quick way to add parallelization to your program.

The parts that are parallelized were taken from Algorithm 2 of the paper [4, p. 580], as seen in fig. 2.

Algorithm 2: NNDescentFull

Data: dataset V, similarity oracle σ, K, ρ, δ
Result: K-NN list B
begin
  $B[v] \longleftarrow \text{Sample}(V, K) \times \{\langle \infty, true \rangle\} \quad \forall v \in V$
  loop
    parallel for $v \in V$ **do**
      $old[v] \longleftarrow$ all items in $B[v]$ with a false flag
      $new[v] \longleftarrow \rho K$ items in $B[v]$ with a true flag
      Mark sampled items in $B[v]$ as false;
1   $old' \leftarrow \text{Reverse}(old)$, $new' \leftarrow \text{Reverse}(new)$
    $c \longleftarrow 0$ //update counter
    parallel for $v \in V$ **do**
      $old[v] \longleftarrow old[v] \cup \text{Sample}(old'[v], \rho K)$
      $new[v] \longleftarrow new[v] \cup \text{Sample}(new'[v], \rho K)$
      for $u_1, u_2 \in new[v], u_1 < u_2$
      or $u_1 \in new[v], u_2 \in old[v]$ **do**
        $l \longleftarrow \sigma(u_1, u_2)$
        // c and $B[.]$ are synchronized.
2       $c \longleftarrow c + \text{UpdateNN}(B[u_1], \langle u_2, l, true \rangle)$
3       $c \longleftarrow c + \text{UpdateNN}(B[u_2], \langle u_1, l, true \rangle)$
    return B **if** $c < \delta NK$

Figure 2: The full NN-Descent algorithm as presented in [4].

3 Experiments

The datasets used both for the development and the following experiments are the ones of the ACM SIGMOD 2023 competition, that are hosted on [2]. On all experiments, the distance function used is the euclidean distance. A lot of experiments were conducted for a lot of different parameter combinations. But, our conclusion is that the biggest bottleneck (at least in our implementation) is the K. Our $B[v]$'s are implemented as a Priority Queue. Even though it serves us by keeping the K neighbors sorted, the problem arises when we try to insert a new neighbor. In `updateNN`, when we insert a new neighbor to see if it belongs in the K nearest, we also check that it isn't already there. But since the internals of the p. queue are obfuscated and its array is not sorted, our only option is to linearly search for it. This gives a complexity of $O(K)$ for each vertex, making searches for large K slow.

3.1 Performance Measures

As Dong, Moses, and Li [4], we also use the *recall* to test our library's performance:

> The ground truth is true K-NN obtained by scanning the datasets in brute force. The recall of one object is the number of its true K-NN members found divided by K. The recall of an approximate K-NNG is the average recall of all objects.

3.2 Default Parameters

Unless stated otherwise, the parameters' default values can be seen in tbl. 1. $D = 0$ means that a basic random graph is used as a start, and no RPTs are computed.

Table 1: The parameters used and their default values.

Parameter	Default value
K	-
δ	0.01
ρ	0.5
D	0
T	4

3.3 System Environment

The experiments were conducted on a desktop PC of the following configuration: An AMD Ryzen 5 3600 6-Core Processor with 12 threads and16GB main memory. The operating system used was Fedora Linux 39, with the 6.6.11-200 kernel and GCC 13.2.1.

3.3.1 Performance

Table 2: Results for $K = 100$, $\delta = 0.01$ without the use of RPTs and single-threaded.

Dataset (N)	ρ	Time (ms)	Iterations	Recall
200	0.5	4457	2	99.0001%
	1.0	5736	2	99.0001%
500	0.5	8757	2	98.9996%
	1.0	13188	2	98.9996%
1000 (1)	0.5	16317	2	98.9993%
	1.0	24627	2	98.9993%
5000 (1)	0.5	102537	3	98.853%
	1.0	145606	3	98.991%

4 Known Issues/Future Work

As stated in sec. 3, our heap implementation is not optimal. Right now we are in the middle of migrating to AVL trees for that purpose, though their implementation and integration to the project were not done by the writing of this report.

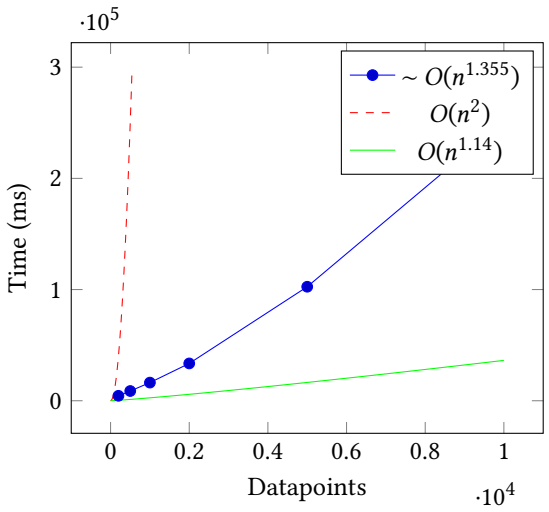

Figure 3: Plot of execution time corresponding to the dataset. Data shown in 2. This plot reinforces the claim that NN-Descent has a complexity of $O(n^{1.14})$ due to its shape.

5 Acknowledgements

We would like to thank our instructor Sarantis Paskalis for his guidance in regards to this project and our colleagues who generously shared tips and findings with us.

References

[1] J. Brugger, *Brj0/nndescent*, Dec. 11, 2023. [Online]. Available: https://github.com/brj0/nndescent.

[2] K. Chousos, *Kchousos/CPPDescent*, Jan. 18, 2024. [Online]. Available: https://github.com/kchousos/CPPDescent.

[3] S. Dasgupta and Y. Freund, "Random projection trees and low dimensional manifolds," in *Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing*, Victoria British Columbia Canada: ACM, May 17, 2008, pp. 537–546, ISBN: 978-1-60558-047-0. DOI: 10.1145/1374376.1374452. [Online]. Available: https://dl.acm.org/doi/10.1145/1374376.1374452.

[4] W. Dong, C. Moses, and K. Li, "Efficient k-nearest neighbor graph construction for generic similarity measures," in *Proceedings of the 20th International Conference on World Wide Web*, ser. WWW '11, New York, NY, USA: Association for Computing Machinery, Mar. 28, 2011, pp. 577–586, ISBN: 978-1-4503-0632-4. DOI: 10.1145/1963405.1963487. [Online]. Available: https://dl.acm.org/doi/10.1145/1963405.1963487.

[5] Enthought, director, *PyNNDescent Fast Approximate Nearest Neighbor Search with Numba | SciPy 2021*, Jul. 23, 2021. [Online]. Available: https://www.youtube.com/watch?v=xPadY4_kt3o.

[6] *Google/googletest*, Google, Jan. 18, 2024. [Online]. Available: https://github.com/google/googletest.

[7] "GSL - GNU Scientific Library - GNU Project - Free Software Foundation." (), [Online]. Available: https://www.gnu.org/software/gsl/.

[8] "How PyNNDescent works — pynndescent 0.5.0 documentation." (), [Online]. Available: https://pynndescent.readthedocs.io/en/latest/how_pynndescent_works.html.

[9] Γ. Ιωαννίδης. "Ανάπτυξη Λογισμικού για Πληροφοριακά Συστήματα." (), [Online]. Available: https://eclass.uoa.gr/courses/DI485/.

[10] D. Kluser, J. Bokstaller, S. Rutz, and T. Buner. "Fast Single-Core K-Nearest Neighbor Graph Computation." arXiv: 2112.06630 [cs, math]. (Dec. 13, 2021), [Online]. Available: http://arxiv.org/abs/2112.06630, preprint.

[11] OpenMP Architecture Review Board, *OpenMP application program interface version 5.2*, Nov. 2021. [Online]. Available: https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf.

[12] Stitch Fix Multithreaded, director, *Algo Hour - Nearest Neighbor Descent (and friends) | Dr. Leland McInnes*, May 22, 2020. [Online]. Available: https://www.youtube.com/watch?v=OvT2NY_FV_g.